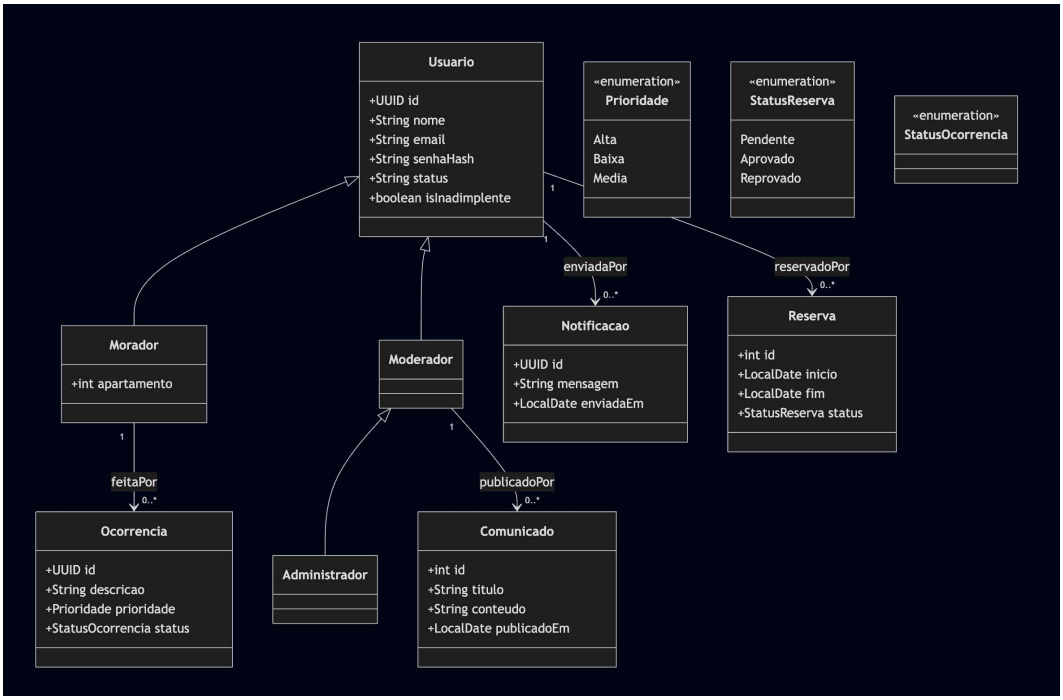
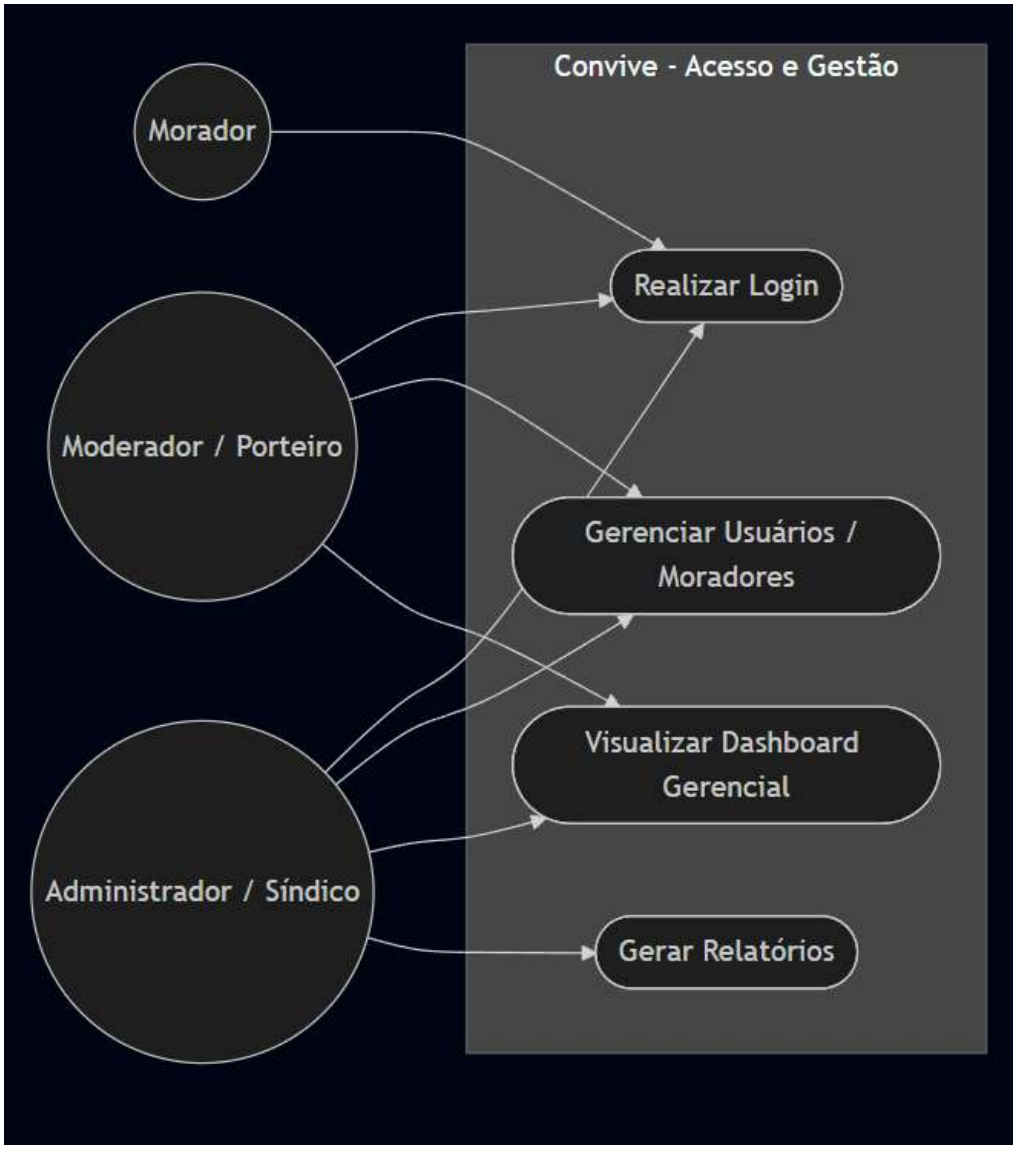
# Diagramas

## Diagrama de classes

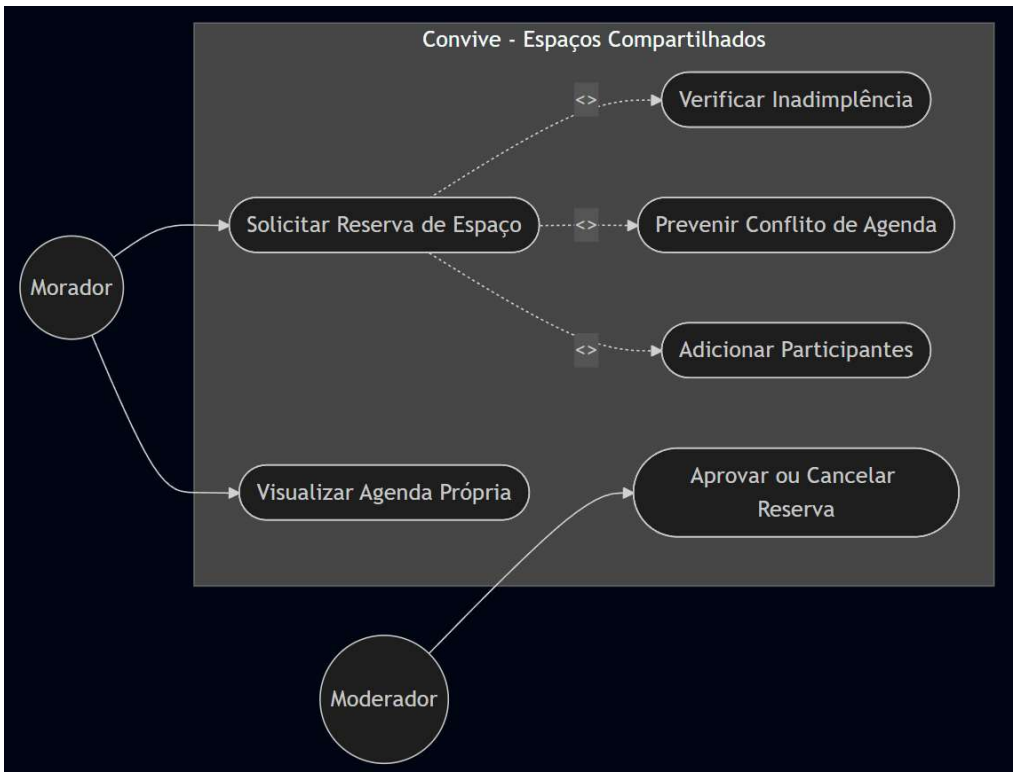



## Diagramas de caso de uso

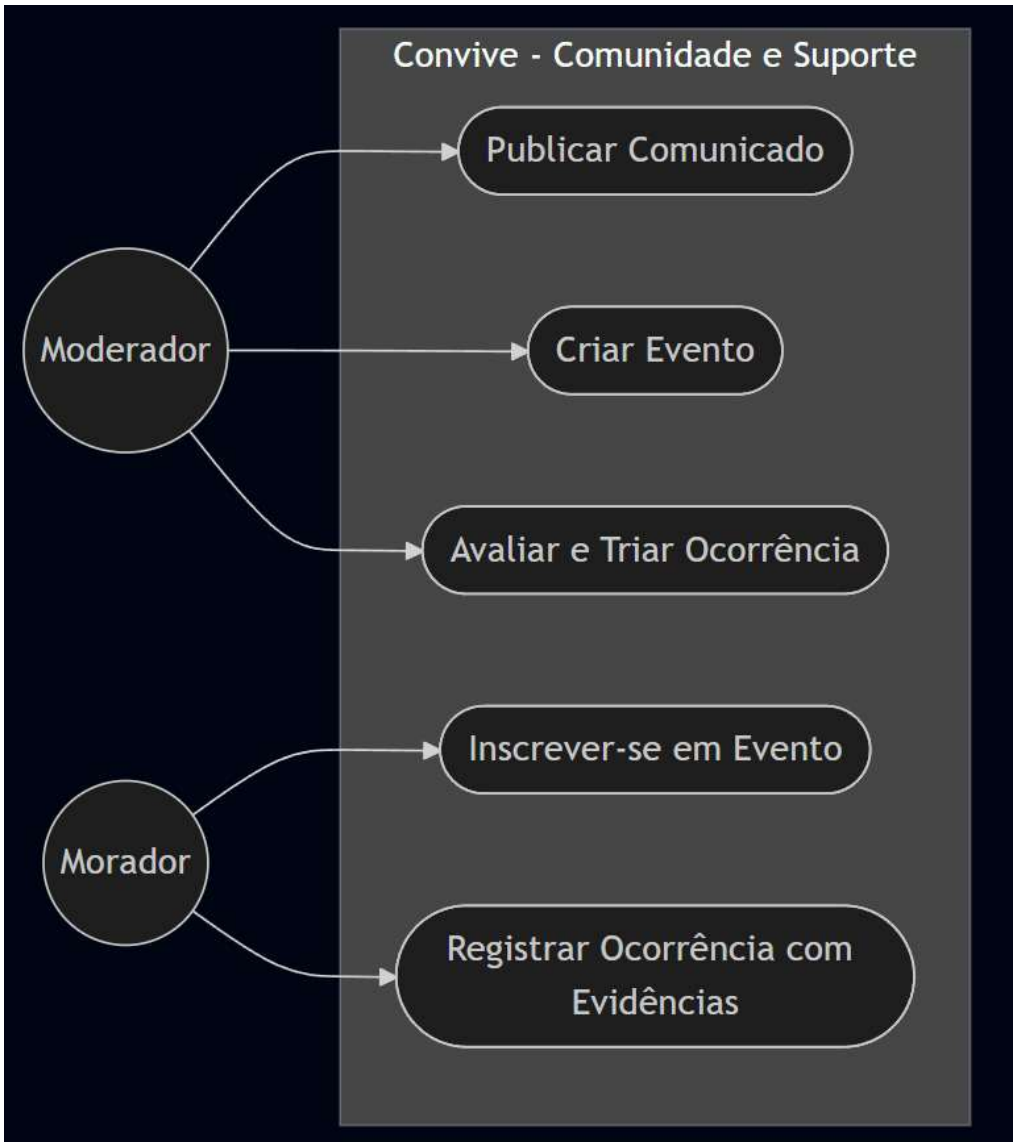
- Acesso / gestão de usuários



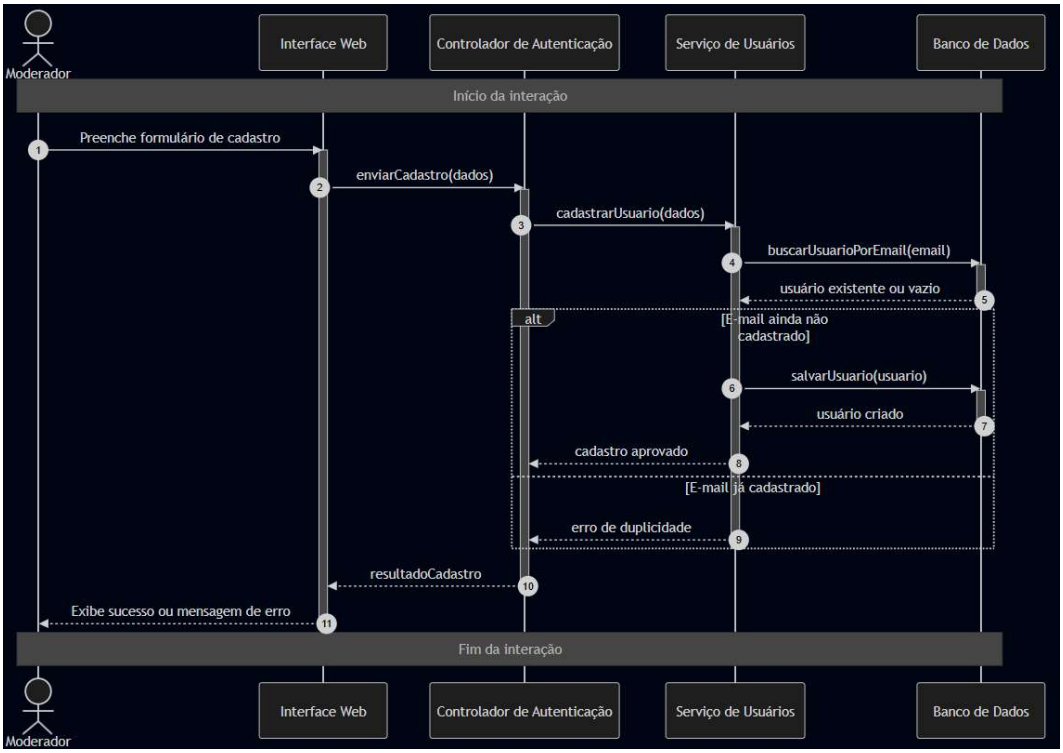

- Reserva de espaços



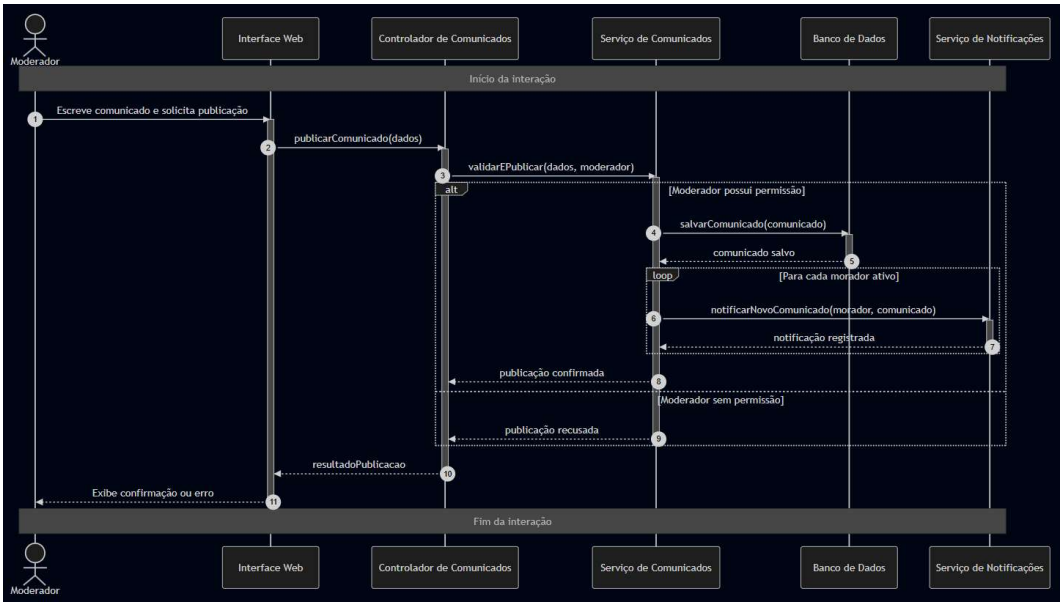
- Ocorrências / Comunicados



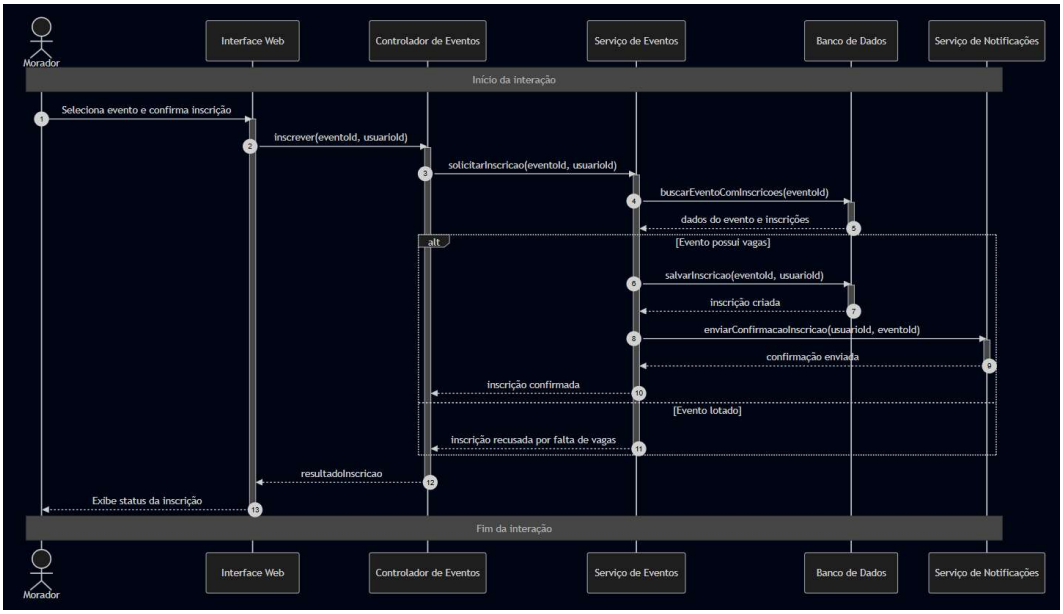

# Diagrama cadastro de usuário



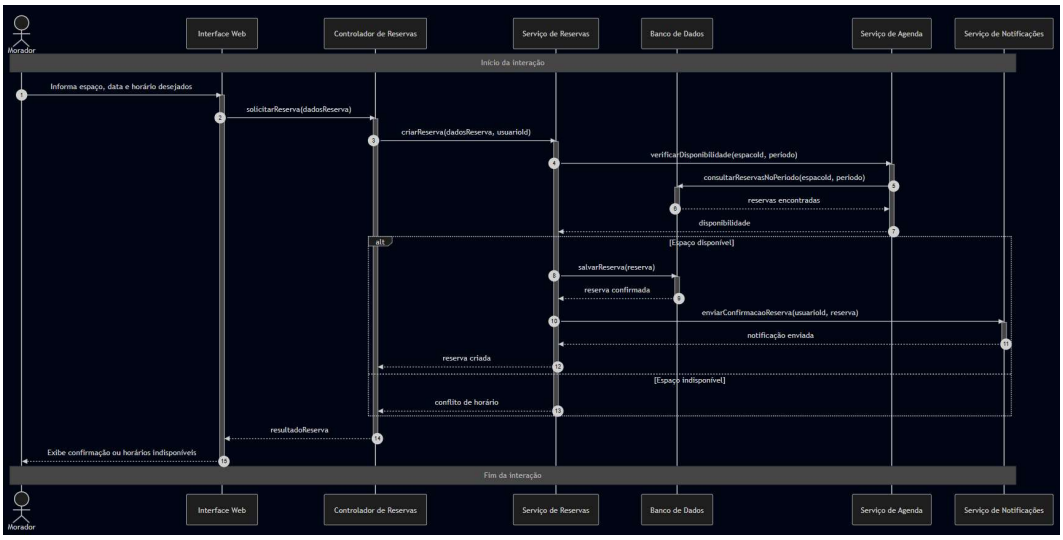

# Diagrama publicação de comunicado



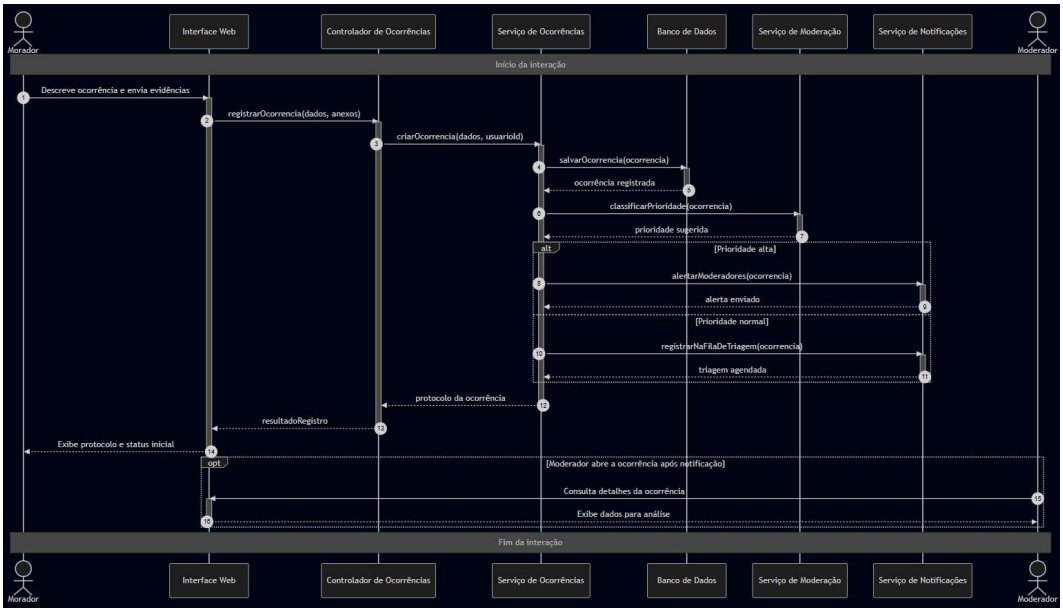

# Diagrama inscrição em evento




# Diagrama reserva de espaço




# Diagrama cadastro de ocorrências



# Diagrama de classes completo

```
classDiagram
    class Usuario {
        +UUID id
        +String nome
        +String email
        +String senhaHash
        +String status
        +boolean isInadimplente
    }

    class Morador {
        +int apartamento
    }

    class Moderador {
    }

    class Administrador {
    }

    class Comunicado {
        +int id
        +String titulo
        +String conteudo
        +LocalDate publicadoEm
    }

    class Notificacao {
        +UUID id
        +String mensagem
        +LocalDate enviadaEm
    }

    class Ocorrencia {
        +UUID id
        +String descricao
        +Prioridade prioridade
        +StatusOcorrencia status
    }

    class Reserva {
        +int id
        +LocalDate inicio
        +LocalDate fim
        +StatusReserva status
    }

    class Prioridade {
```

```
        <<enumeration>>
        Alta
        Baixa
        Media
    }

    class StatusReserva {
        <<enumeration>>
        Pendente
        Aprovado
        Reprovado
    }

    class StatusOcorrencia {
        <<enumeration>>
    }

    Usuario <|-- Morador
    Usuario <|-- Moderador
    Moderador <|-- Administrador

    Moderador "1" --> "0..*" Comunicado : publicadoPor
    Morador "1" --> "0..*" Ocorrencia : feitaPor
    Usuario "1" --> "0..*" Notificacao : enviadaPor
    Usuario "1" --> "0..*" Reserva : reservadoPor
```

# Diagrama de cadastro de usuário

```
sequenceDiagram
    autonumber
    actor Moderador
    participant WebApp as Interface Web
    participant AuthController as Controlador de Autenticação
    participant UsuarioService as Serviço de Usuários
    participant Banco as Banco de Dados

    Note over Moderador,Banco: Início da interação
    Moderador->>WebApp: Preenche formulário de cadastro
    activate WebApp
    WebApp->>AuthController: enviarCadastro(dados)
    activate AuthController
    AuthController->>UsuarioService: cadastrarUsuario(dados)
    activate UsuarioService
    UsuarioService->>Banco: buscarUsuarioPorEmail(email)
    activate Banco
    Banco-->>UsuarioService: usuário existente ou vazio
    deactivate Banco
```

```

    alt E-mail ainda não cadastrado
        UsuarioService->>Banco: salvarUsuario(usuario)
        activate Banco
        Banco-->>UsuarioService: usuário criado
        deactivate Banco
        UsuarioService-->>AuthController: cadastro aprovado
    else E-mail já cadastrado
        UsuarioService-->>AuthController: erro de duplicidade
    end

    deactivate UsuarioService
    AuthController-->>WebApp: resultadoCadastro
    deactivate AuthController
    WebApp-->>Moderador: Exibe sucesso ou mensagem de erro
    deactivate WebApp
    Note over Moderador,Banco: Fim da interação
```

# Diagrama publicação de comunicado para a comunidade

```
sequenceDiagram
    autonumber
    actor Moderador
    participant WebApp as Interface Web
    participant ComunicadoController as Controlador de Comunica
    participant ComunicadoService as Serviço de Comunicados
    participant Banco as Banco de Dados
    participant NotificacaoService as Serviço de Notificações

    Note over Moderador,NotificacaoService: Início da interação
    Moderador->>WebApp: Escreve comunicado e solicita publicaçã
    activate WebApp
    WebApp->>ComunicadoController: publicarComunicado(dados)
    activate ComunicadoController
    ComunicadoController->>ComunicadoService: validarEPublicar(
    activate ComunicadoService

    alt Moderador possui permissão
        ComunicadoService->>Banco: salvarComunicado(comunicado)
        activate Banco
        Banco-->>ComunicadoService: comunicado salvo
        deactivate Banco

        loop Para cada morador ativo
            ComunicadoService->>NotificacaoService: notificarNo
```

```
            activate NotificacaoService
            NotificacaoService-->>ComunicadoService: notificaçã
            deactivate NotificacaoService
        end

        ComunicadoService-->>ComunicadoController: publicação c
    else Moderador sem permissão
        ComunicadoService-->>ComunicadoController: publicação r
    end

    deactivate ComunicadoService
    ComunicadoController-->>WebApp: resultadoPublicacao
    deactivate ComunicadoController
    WebApp-->>Moderador: Exibe confirmação ou erro
    deactivate WebApp
    Note over Moderador,NotificacaoService: Fim da interação
```

# Diagrama inscricao de morador em eventos

```
sequenceDiagram
    autonumber
    actor Morador
    participant WebApp as Interface Web
    participant EventoController as Controlador de Eventos
    participant EventoService as Serviço de Eventos
    participant Banco as Banco de Dados
    participant NotificacaoService as Serviço de Notificações

    Note over Morador,NotificacaoService: Início da interação
    Morador->>WebApp: Seleciona evento e confirma inscrição
    activate WebApp
    WebApp->>EventoController: inscrever(eventoId, usuarioId)
    activate EventoController
    EventoController->>EventoService: solicitarInscricao(evento
    activate EventoService
    EventoService->>Banco: buscarEventoComInscricoes(eventoId)
    activate Banco
    Banco-->>EventoService: dados do evento e inscrições
    deactivate Banco

    alt Evento possui vagas
        EventoService->>Banco: salvarInscricao(eventoId, usuari
        activate Banco
        Banco-->>EventoService: inscrição criada
        deactivate Banco
```

```
        EventoService->>NotificacaoService: enviarConfirmacaoIn
        activate NotificacaoService
        NotificacaoService-->>EventoService: confirmação enviad
        deactivate NotificacaoService
        EventoService-->>EventoController: inscrição confirmada
    else Evento lotado
        EventoService-->>EventoController: inscrição recusada p
    end

    deactivate EventoService
    EventoController-->>WebApp: resultadoInscricao
    deactivate EventoController
    WebApp-->>Morador: Exibe status da inscrição
    deactivate WebApp
    Note over Morador,NotificacaoService: Fim da interação
```

# Diagrama de reserva de espaços

```
sequenceDiagram
    autonumber
    actor Morador
    participant WebApp as Interface Web
    participant ReservaController as Controlador de Reservas
    participant ReservaService as Serviço de Reservas
    participant Banco as Banco de Dados
    participant AgendaService as Serviço de Agenda
    participant NotificacaoService as Serviço de Notificações

    Note over Morador,NotificacaoService: Início da interação
    Morador->>WebApp: Informa espaço, data e horário desejados
    activate WebApp
    WebApp->>ReservaController: solicitarReserva(dadosReserva)
    activate ReservaController
    ReservaController->>ReservaService: criarReserva(dadosReser
    activate ReservaService
    ReservaService->>AgendaService: verificarDisponibilidade(es
    activate AgendaService
    AgendaService->>Banco: consultarReservasNoPeriodo(espacoId,
    activate Banco
    Banco-->>AgendaService: reservas encontradas
    deactivate Banco
    AgendaService-->>ReservaService: disponibilidade
    deactivate AgendaService

    alt Espaço disponível
        ReservaService->>Banco: salvarReserva(reserva)
        activate Banco
```

```
        Banco-->>ReservaService: reserva confirmada
        deactivate Banco
        ReservaService->>NotificacaoService: enviarConfirmacaoR
        activate NotificacaoService
        NotificacaoService-->>ReservaService: notificação envia
        deactivate NotificacaoService
        ReservaService-->>ReservaController: reserva criada
    else Espaço indisponível
        ReservaService-->>ReservaController: conflito de horári
    end

    deactivate ReservaService
    ReservaController-->>WebApp: resultadoReserva
    deactivate ReservaController
    WebApp-->>Morador: Exibe confirmação ou horários indisponív
    deactivate WebApp
    Note over Morador,NotificacaoService: Fim da interação
```

# Diagrama cadastro de ocorrencias

```
sequenceDiagram
    autonumber
    actor Morador
    participant WebApp as Interface Web
    participant OcorrenciaController as Controlador de Ocorrênc
    participant OcorrenciaService as Serviço de Ocorrências
    participant Banco as Banco de Dados
    participant ModeracaoService as Serviço de Moderação
    participant NotificacaoService as Serviço de Notificações
    actor Moderador

    Note over Morador,Moderador: Início da interação
    Morador->>WebApp: Descreve ocorrência e envia evidências
    activate WebApp
    WebApp->>OcorrenciaController: registrarOcorrencia(dados, a
    activate OcorrenciaController
    OcorrenciaController->>OcorrenciaService: criarOcorrencia(d
    activate OcorrenciaService
    OcorrenciaService->>Banco: salvarOcorrencia(ocorrencia)
    activate Banco
    Banco-->>OcorrenciaService: ocorrência registrada
    deactivate Banco

    OcorrenciaService->>ModeracaoService: classificarPrioridade
    activate ModeracaoService
    ModeracaoService-->>OcorrenciaService: prioridade sugerida
```

```
    deactivate ModeracaoService

    alt Prioridade alta
        OcorrenciaService->>NotificacaoService: alertarModerado
        activate NotificacaoService
        NotificacaoService-->>OcorrenciaService: alerta enviado
        deactivate NotificacaoService
    else Prioridade normal
        OcorrenciaService->>NotificacaoService: registrarNaFila
        activate NotificacaoService
        NotificacaoService-->>OcorrenciaService: triagem agenda
        deactivate NotificacaoService
    end

    OcorrenciaService-->>OcorrenciaController: protocolo da oco
    deactivate OcorrenciaService
    OcorrenciaController-->>WebApp: resultadoRegistro
    deactivate OcorrenciaController
    WebApp-->>Morador: Exibe protocolo e status inicial
    deactivate WebApp

    opt Moderador abre a ocorrência após notificação
        Moderador->>WebApp: Consulta detalhes da ocorrência
        activate WebApp
        WebApp-->>Moderador: Exibe dados para análise
        deactivate WebApp
    end

    Note over Morador,Moderador: Fim da interação
```

# Diagrama caso de uso Ocorrências

```
flowchart LR
    Mor((Morador))
    Mod((Moderador))

    subgraph Convive - Comunidade e Suporte
        UC12([Publicar Comunicado])
        UC13([Criar Evento])
        UC14([Inscrever-se em Evento])
        UC15([Registrar Ocorrência com Evidências])
        UC16([Avaliar e Triar Ocorrência])
    end

    Mod --> UC12
    Mod --> UC13
    Mor --> UC14
```

```

    Mor --> UC15
    Mod --> UC16
```

# Diagrama caso de uso Reservas

```
flowchart LR
    Mor((Morador))
    Mod((Moderador))

    subgraph Convive - Espaços Compartilhados
        UC6([Solicitar Reserva de Espaço])
        UC7([Verificar Inadimplência])
        UC8([Prevenir Conflito de Agenda])
        UC9([Adicionar Participantes])
        UC10([Visualizar Agenda Própria])
        UC11([Aprovar ou Cancelar Reserva])
    end

    Mor --> UC6
    Mor --> UC10

    %% Relacionamentos de Inclusão e Extensão
    UC6 -. "<<include>>" .-> UC7
    UC6 -. "<<include>>" .-> UC8
    UC6 -. "<<extend>>" .-> UC9

    Mod --> UC11
```

# Diagrama caso de uso Acessos

```
flowchart LR
    %% Atores
    Mor((Morador))
    Mod((Moderador / Porteiro))
    Adm((Administrador / Síndico))

    subgraph Convive - Acesso e Gestão
        UC2([Realizar Login])
        UC3([Gerenciar Usuários / Moradores])
        UC4([Visualizar Dashboard Gerencial])
        UC5([Gerar Relatórios])
    end
```

```

    Mor --> UC2
    Mod --> UC2
    Adm --> UC2

    Adm --> UC3
    Mod --> UC3
    Mod --> UC4
    Adm --> UC4
    Adm --> UC5
```